

Software Engineering Transition Plan (Overview for Mentor)

Time Commitment: 3-5 hours/week (~15-20 hours/month)

Current Role: Analytics Engineer (4 years, dbt/Airflow/SQL) → **Target:** Software Engineer

Strategy: One well-rounded portfolio project (Full-Stack Expense Tracker) → Optimizely Team

Current Foundation

Strengths: SQL/data modeling, Python fundamentals, dbt, Airflow, stakeholder management

Learning: Web APIs, frontend, testing/CI/CD, architecture patterns

Approach: Backend-first, depth over breadth, iterative refactor, weekly mentor check-ins

Project: Full-Stack Expense Tracker (CLI → Web App)

Tech Stack: Flask/Jinja2 + React (both frontends) → FastAPI → SQLite/SQLAlchemy

Architecture Note: Both frontends (Flask and React) coexist, sharing the same FastAPI backend and SQLite database.

Phase 1: Backend Foundations (Weeks 1-4, ~12-20 hours)

Week 1-2: HTTP, REST APIs & FastAPI Fundamentals

Goal: Build first working API with automatic documentation

Core Concepts: - HTTP methods (GET, POST, PUT, DELETE), status codes, request/response cycle - JSON serialization, RESTful principles - FastAPI routing, path operations, Pydantic models for validation - In-memory CRUD storage (expenses list)

Deliverable: Working API with /docs, basic CRUD endpoints tested via browser/curl

Week 3-4: Databases & ORMs

Goal: Replace in-memory storage with persistent database

Core Concepts: - SQLAlchemy ORM: models, sessions, queries, relationships - Alembic migrations (schema versioning) - Transactions and ACID properties - ORM vs raw SQL tradeoffs

Deliverable: DB-backed CRUD with proper schema, basic relationships (User → Expenses)

Phase 2: Frontend Development (Weeks 5-12, ~24-40 hours)

Week 5-6: Flask Frontend + Jinja2

Goal: Build functional UI that consumes FastAPI backend

Core Concepts: - Flask routing, Jinja2 templating (inheritance, macros, partials) - HTML forms, semantic structure - CSS basics (selectors, flexbox) - Server-side rendering vs client-side rendering

Deliverable: Working web UI (list expenses, add/edit forms, end-to-end flow verified)

Week 7-8: JavaScript Fundamentals

Goal: Learn the language of the web before diving into React

Core Concepts: - Variables, functions, arrow functions, callbacks - ES6+ features: destructuring, spread operator, template literals, modules - Promises and async/await - DOM manipulation basics, JSON parsing

Deliverable: JavaScript exercises fetching data from FastAPI endpoints using fetch()

Week 9-10: React Basics

Goal: Understand component-based UI development

Core Concepts: - JSX syntax, functional components, props, children - State management with useState, side effects with useEffect - Event handling, conditional rendering, lists - Component composition (parent/child data flow)

Deliverable: Static React components (ExpenseItem, ExpenseList, ExpenseForm) with local state

Week 11-12: React + FastAPI Integration

Goal: Build the Expense Tracker React frontend connected to existing API

Core Concepts: - Fetching data using fetch() or axios, loading/error states - CORS configuration, environment variables - API service layer (separate fetch logic from components) - Controlled forms, validation

Deliverable: Full CRUD React frontend sharing same FastAPI backend as Flask version

Phase 3: Software Design, Auth & Testing (Weeks 13-18, ~18-30 hours)

Week 13-14: Software Design Fundamentals

Goal: Refactor for maintainability and production readiness

Core Concepts: - Layered architecture: routes → services → data access - Error handling, input validation (Pydantic validators) - Configuration management (env variables), logging - Dependency Injection, Repository Pattern

Deliverable: Clean architecture, proper error responses, logging for debugging

Week 15-16: Authentication & Authorization

Goal: Add user management and secure endpoints

Core Concepts: - JWT authentication (registration, login, password hashing with bcrypt) - Auth vs authz, session-based vs token-based - OWASP Top 10 awareness (SQL injection, XSS, CSRF) - Protected endpoints (users see only their expenses)

Deliverable: User registration/login, JWT-protected API, password reset flow

Week 17-18: Testing & CI/CD

Goal: Automated testing and deployment pipelines

Core Concepts: - Unit tests vs integration tests, pytest (fixtures, mocking, parametrize) - TestClient for FastAPI endpoints - GitHub Actions (run tests on push/PR, linting) - Test coverage targets (>70% on critical paths)

Deliverable: Test suite passing, CI green on GitHub, coverage report

Phase 4: Advanced Features (Weeks 19-22, ~12-20 hours)

Week 19-22: Multi-Tenancy + RBAC + API Keys + CI/CD

Goal: Add enterprise SaaS patterns to Expense Tracker

Core Concepts: - Multi-tenancy: organization/workspace data isolation (tenant_id scoping) - RBAC: Owner/Admin/Member roles with permission middleware - API key generation/storage (hashed), validation, revocation - Usage tracking (API calls per org/user), rate limiting - GitHub Actions CI/CD pipelines

Deliverable: Multi-org Expense Tracker with role-based permissions, API key access, CI/CD pipeline

Phase 5: Architecture & Scale (Weeks 23-26, ~12-20 hours)

Week 23-24: Architecture Patterns

Goal: Think beyond single apps to system design

Core Concepts: - Monolith vs Microservices vs SCS (Self-Contained Systems) - Event-driven architecture (message queues, pub/sub) - API Gateway pattern, database patterns (read replicas, caching, sharding concepts)

Deliverable: Architecture diagrams for Expense Tracker decomposition options

Week 25-26: Design Patterns & Clean Code

Goal: Write professional-grade maintainable code

Core Concepts: - Design patterns: Singleton, Factory, Adapter, Strategy, Observer - SOLID principles, DRY vs WET - Code review practices, refactoring techniques

Deliverable: Refactored codebase with documented patterns, improved readability

Phase 6: Deployment & Team Readiness (Weeks 27-28, ~6-10 hours)

Week 27-28: Deployment & Team Readiness

Goal: Deploy to production, prepare for team collaboration

Core Concepts: - Docker basics, Terraform IaC (provision/deploy/destroy) - Git collaboration (branching strategies, PRs, code review, commit messages) - Documentation (READMEs, API docs beyond auto-generated) - Team workflow (tickets, Definition of Done, progress updates)

Deliverable: Deployed app (cloud), comprehensive docs, team workflow artifacts

Key Questions for Mentor

1. **Phase Ordering:** Correct sequence for 3-5 hrs/week pace?
 2. **MVP Scope:** Minimum for Phase 1-2 to prove end-to-end value quickly?
 3. **FastAPI Structure:** Best practices for routers/services/repositories from Day 1?
 4. **Testing Strategy:** Which tests are critical early (unit vs integration)?
 5. **Optimizely Alignment:** Which topics matter most — auth, multi-tenancy, CI/CD, architecture?
-

Review Checkpoints

Checkpoint	Deliverable	Mentor Focus
A	FastAPI CRUD + Pydantic + <code>/docs</code>	API design, validation, documentation
B	Flask UI + React UI (both working)	Server-side vs client-side rendering
C	DB-backed CRUD + migrations	ORM usage, schema evolution
D	Auth + tests + CI green	Security, test coverage, automation
E	Multi-tenancy + RBAC + API keys	Enterprise patterns, data isolation

What I Need from Mentor

- **Code Review:** Structure + correctness at each checkpoint (async preferred)
- **Design Validation:** Data model, endpoint design, auth approach (20-min calls at key milestones)
- **Scope Guidance:** What to cut/defer to maintain momentum with limited time
- **Industry Context:** How does Optimizely approach similar problems?

Next Step: Complete Checkpoint A → Schedule first code review